

ParakhMitra — The Codebook

A phase-by-phase technical textbook for this exact project, written to be read start-to-finish and presented from.

How to read this document

This is written for a whole team, some of whom have never been taught what an **API**, a **database**, **authentication**, or **deployment** actually *are*. So we do two things on every topic, and we never skip either one:

1. **We teach the concept from scratch** — in plain language, with a real-world analogy, and we introduce the precise technical word the first time it appears (in **bold**), so you can use it correctly in the presentation.
2. **We show the real code from this repository that implements it** — with the real file path printed above every snippet, explained line by line, including the actual mechanism underneath (not a hand-wave).

Read it front to back and you'll understand both *how software like this works in general* and *how our ParakhMitra is built*, down to the individual functions. It's deliberately detailed: when we present, any of us should be able to open to a phase and explain exactly what happens and why.

What ParakhMitra does, in one breath: an inspection officer photographs the front and back of a packaged product; the app reads the mandatory legal declarations off the label using AI, checks them against India's Legal Metrology packaging rules, and produces a permanent, tamper-proof inspection record plus a printable government-style notice.

The phases follow the real path of a scan through the system:

1. The two-program architecture (frontend vs backend) and how they talk
2. The frontend — screens, routing, roles, and the scan flow
3. The backend API — endpoints and the full request lifecycle
4. AI extraction with Gemini (and what JSON / structured output really is)
5. The deterministic rule engine (the 8 real rules, in full)
6. Data, authentication & security (database, JWT, RLS, the real SQL)
7. The PDF Improvement Notice (templating and HTML->PDF)
8. Deployment — putting it live on the internet

A vocabulary you'll see throughout: **client** = the program making a request (our frontend/phone); **server** = the program answering (our backend); **request/response** = one round trip between them.

Phase 1 — Two programs that talk over the internet

1. The concept

Imagine a restaurant. There's a **dining room** where guests sit, read the menu, and place orders, and a **kitchen** where the actual cooking happens and where the expensive knives and secret recipes live. Guests never walk into the kitchen; a **waiter** carries a written order in and a finished plate back.

Our project is built as exactly these **two separate programs**:

- The **frontend** (the dining room) runs inside the web browser on the officer's phone — the buttons, the camera screen, the results page. It's friendly and public; it holds no secrets.
- The **backend** (the kitchen) runs on a server computer far away. It holds the secret keys (the AI key, the master database key), talks to the AI, and enforces the rules. No phone can see inside it.

The waiter between them is an **HTTP request**. **HTTP** (HyperText Transfer Protocol) is simply the agreed language browsers and servers use to talk over the internet — a request goes out, a response comes back. The backend exposes an **API** (Application Programming Interface): a fixed, published list of operations it's willing to perform, exactly like a menu. The frontend never scans a package itself — it sends an order ("here are two photo filenames, please scan them") to the backend's API and waits for the plate.

One more foundational rule you must know for the presentation: browsers enforce the **same-origin policy**. A web page loaded from address A is, by default, *not allowed* to call a server at a different address B. Since our dining room (frontend) and kitchen (backend) live at different addresses, the kitchen has to explicitly say "I permit orders from that specific dining room." That permission mechanism is called **CORS** (Cross-Origin Resource Sharing). We meet it properly in Phase 8, but it's introduced here because it's fundamental to *why two programs at different addresses can talk at all*.

In our repository the two programs are two folders you can literally see: `frontend/` and `backend/`.

2. What we actually built

The frontend must know the kitchen's address. That lives in one constant:

`frontend/src/lib/api.ts`

```
const API_BASE_URL =
  (import.meta.env.VITE_API_BASE_URL as string | undefined)?.replace(/\/$/, '') ||
  'http://127.0.0.1:8000'
```

Line by line: `import.meta.env.VITE_API_BASE_URL` reads a value that was injected when the app was built (an **environment variable** — a setting supplied from outside the code; Phase 8). `?.replace(/\/$/, '')` strips a trailing slash if present, so we never accidentally build a double-slash URL. The `|| 'http://127.0.0.1:8000'` is a fallback: if nobody set the variable, assume the backend is on *this same machine* (`127.0.0.1` literally means "localhost") at **port 8000** (a port is a numbered doorway on a computer; a machine can run many servers, each on its own port). So on a developer's laptop this points at the local backend; in production it points at the real internet address.

On the kitchen side, the server opens the door to that dining room and plugs in its menu:

backend/app/main.py

```
app = FastAPI(
    title="Legal Metrology Label Compliance Checker API",
    ...
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(scans_router)
```

`FastAPI(...)` creates the server application object. `add_middleware(CORSMiddleware, ...)` installs a **middleware** — code that inspects every request as it comes in, before it reaches the actual endpoint. This particular middleware implements the CORS permission: `allow_origins=settings.CORS_ORIGINS` is the guest-list of dining rooms allowed to order (a list we configure per environment), `allow_methods=["*"]` and `allow_headers=["*"]` permit all HTTP verbs and request headers from those allowed origins, and `allow_credentials=True` lets the browser send along the login token. `include_router(scans_router)` attaches the actual scan/notice menu (Phase 3).

The kitchen also publishes a tiny "are you alive?" endpoint, which we'll rely on for both monitoring and deployment:

backend/app/main.py

```
@app.get("/health", tags=["system"], summary="Health check endpoint")
def health_check():
    return {
        "status": "healthy",
        "service": "legal-metrology-backend",
        "gemini_model": settings.GEMINI_MODEL,
        "storage_bucket": settings.STORAGE_BUCKET,
    }
```

`@app.get("/health")` says "when someone sends an HTTP **GET** (a read-only request) to the address `/health`, run this function." It returns a small dictionary, which FastAPI automatically turns into a JSON response. Anyone (a monitoring tool, our hosting platform, us in a browser) can hit this to confirm the backend is running and see which AI model and storage bucket it's wired to.

3. Why we did it this way

The split into two programs comes down to **secrets and trust**. The backend holds a *master key* that can read and write everyone's data and spend money on the AI. If that key lived in the phone app, anyone could extract it from the downloaded code and abuse it — there is no way to keep a secret inside software running on someone else's device. By keeping the powerful keys in the kitchen and handing the phone only a limited public menu, a stolen phone or a hostile user can't cause damage. The split also lets each half be built, updated, scaled, and hosted independently — we deploy the dining room to Vercel and the kitchen to Render

(Phase 8), and either can be redeployed without touching the other. The price we pay is the CORS/URL wiring, which is a small, well-understood cost.

Phase 2 — The frontend: screens, routing, roles, and the scan flow

1. The concept

The frontend is the part you see and touch. Think of it as a **building with many rooms**; each room (login, dashboard, new-scan, history) is a **screen**. Something must decide which room you're in based on the sign on the door — the web address (`/scan`, `/history`) — and that job is **routing**.

Rooms are assembled from reusable pieces called **components**: a component is a self-contained, reusable brick of interface (a button, a photo tile, an entire screen) that can be nested and reused. Our components are written in **React**, the most widely used library for building component-based interfaces, and in **TypeScript** — ordinary JavaScript with *type labels* on the data. A **type** is a promise about the shape of a value ("this is a number," "this is a `File`"); TypeScript checks those promises before the app runs, catching a whole class of bugs early. The bundler that compiles all this into files a browser can load is **Vite**.

Two more essential ideas. **State** is a component's live memory of "what's happening right now" — which photo was just taken, whether an upload is in progress, whether an error is showing. When state changes, React automatically **re-renders** (redraws) the affected part of the screen to match. And **context** is a way to share one piece of state (like "who is logged in") across the *whole* app without passing it hand-to-hand through every component — we use it for authentication.

2. What we actually built

Everything boots from one file that mounts the app into the page and wraps it in the two shared systems it needs:

`frontend/src/main.tsx`

```
createRoot(document.getElementById('root')!).render(  
  <StrictMode>  
    <BrowserRouter>  
      <AuthProvider>  
        <App />  
      </AuthProvider>  
    </BrowserRouter>  
  </StrictMode>,  
)
```

`createRoot(...).render(...)` is React attaching itself to the empty `<div id="root">` in the HTML page and taking over from there. The nesting matters, read outside-in: `BrowserRouter` is the router — it reads the URL and provides routing to everything inside it. `AuthProvider` is the shared login **context** (built in Phase 6) — every component under it can ask "who's logged in, and what's their role?". `<App />` is the building itself. `StrictMode` is a development-only helper that surfaces bugs.

`App` is the receptionist that decides what you may see:

`frontend/src/App.tsx`

```

export default function App() {
  const { session, loading, accessDenied, isAdmin } = useAuth()

  if (loading) return <FullScreenLoader label="Loading ParakhMitra..." />
  if (!session) return <Login />
  if (accessDenied) return <AccessDenied />

  return (
    <Routes>
      <Route path="/scan/:id" element={<ScanDetail />} />
      <Route element={<Layout />>
        <Route path="/" element={<Dashboard />} />
        <Route path="/scan" element={isAdmin ? <Navigate to="/" replace /> : <NewScan />} />
        <Route path="/results" element={isAdmin ? <Navigate to="/" replace /> : <Results />} />
        <Route path="/history" element={<History />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/users" element={<Users />} />
      </Route>
      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  )
}

```

`const { session, loading, accessDenied, isAdmin } = useAuth()` pulls four facts out of the shared login context. The three `if` lines are gates in order: while we're still checking who you are, show a spinner; if there's no session (not logged in), show `Login`; if you're logged in but not authorised, show `AccessDenied`. Only past all three do we reach `<Routes>`, the list of door-signs. Each `<Route path=... element=...>` maps a URL to a screen. `/scan/:id` (the `:id` is a wildcard — any scan's id) is deliberately placed *outside* `<Layout>` because the detail screen has its own top bar. Everything else is nested inside `<Layout>` so it shares the app frame and bottom navigation. Note the role enforcement written directly into the routes: `element={isAdmin ? <Navigate to="/" replace /> : <NewScan />}` means "if an admin lands on `/scan`, redirect them home." The final `path="*" element={<Navigate to="/" replace />}` catches any unknown address and sends it home.

The shared frame, `Layout`, also decides which buttons appear in the bottom navigation based on role:

frontend/src/components/Layout.tsx

```

{/* Center scan action – officers only. Admins are supervisors and do not scan. */}
{!isAdmin && (
  <NavLink to="/scan" className={`navitem navitem--fab ...`} aria-label="New scan">
    <span className="navitem__fab"><ScanIcon size={24} /></span>
  </NavLink>
)}

{isAdmin && (
  <NavLink to="/users" className={...}>
    <UsersIcon className="navitem__icon" size={22} />
    Users
  </NavLink>
)}

```

`{!isAdmin && (...)}` is React's conditional rendering: the expression before `&&` must be true for the element after it to appear. So the big central **Scan** button renders *only for non-admins* (officers), and the **Users** management tab renders *only for admins*. This is why the two roles literally see different navigation bars — and, crucially, it's backed up by the route-level redirect above, so hiding the button isn't the only defence.

Now the heart of the app — the **scan flow**, in the New Scan screen. First, each photo tile builds a live preview of the chosen image:

frontend/src/screens/NewScan.tsx

```
function CaptureTile({ label, file, onPick }: CaptureTileProps) {
  const preview = useMemo(() => (file ? URL.createObjectURL(file) : null), [file])
  useEffect(() => {
    return () => { if (preview) URL.revokeObjectURL(preview) }
  }, [preview])
  ...
  <input type="file" accept="image/*" capture="environment"
    onChange={(e) => onPick(e.target.files?.[0] ?? null)} />
```

`URL.createObjectURL(file)` turns the picked image `File` into a temporary in-browser URL so we can show a thumbnail without uploading yet. `useMemo(..., [file])` recomputes that URL only when the file changes (not on every re-render). The `useEffect` returns a cleanup function that calls `URL.revokeObjectURL(preview)` to free the memory when the file changes or the tile disappears — a small but real resource-management detail. The `<input type="file" ... capture="environment">` is a standard file picker; `capture="environment"` hints mobile browsers to open the **rear camera** directly, which is what an officer in the field wants.

When both photos exist and the officer taps "Scan," `handleScan` runs the whole client-side flow:

frontend/src/screens/NewScan.tsx

```
async function handleScan() {
  if (!front || !back || !user) return
  setSubmitting(true)
  setError(null)
  try {
    setStage('Uploading package photos...')
    const [frontPath, backPath] = await Promise.all([
      uploadEvidencePhoto(front),
      uploadEvidencePhoto(back),
    ])

    setStage('Extracting declarations & checking rules...')
    const result = await createScan({ frontPath, backPath, userId: user.id, category })

    navigate('/results', { state: { result } })
  } catch (e) {
    setError(e instanceof Error ? e.message : 'Scan failed. Please try again.')
    setSubmitting(false)
    setStage('')
  }
}
```

`async/await` is how JavaScript waits for slow things (network calls) without freezing the screen. It guards against missing inputs, flips `submitting` state on (which swaps the screen to a spinner), then **uploads both photos in parallel** with `Promise.all([...])` — both uploads run at once and we wait for both to finish, getting back the two storage filenames. Then `createScan(...)` places the order to the backend and awaits the result (the plate). `navigate('/results', { state: { result } })` moves to the Results screen, handing the returned data along in the router's state. Any failure in the whole sequence is caught and shown as a friendly error.

The two helper functions it calls live in `api.ts`. Uploading a photo goes *straight to storage*, not through our backend:

frontend/src/lib/api.ts

```
export async function uploadEvidencePhoto(file: File): Promise<string> {
  const path = `${crypto.randomUUID()}.jpg`
  const { error } = await supabase.storage
    .from(EVIDENCE_BUCKET)
    .upload(path, file, { contentType: 'image/jpeg', upsert: false })
  if (error) throw new Error(`Image upload failed: ${error.message}`)
  return path
}
```

`crypto.randomUUID()` generates a **UUID** — a Universally Unique Identifier, a random 128-bit value like `330389d7-8a0d-...` that is effectively guaranteed never to collide with another. We use it as the filename so two officers uploading `photo.jpg` never overwrite each other.

`supabase.storage.from(EVIDENCE_BUCKET).upload(...)` sends the raw image bytes to the online file store (a **bucket** is a named container of files; Phase 6). `upsert: false` means "fail rather than overwrite if that name somehow exists." It returns just the short filename — that tiny string, not the megabytes of image, is what travels to our backend.

And placing the scan order:

frontend/src/lib/api.ts

```
const res = await fetch(`${API_BASE_URL}/api/scans`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', ...(await authHeaders()) },
  body: JSON.stringify({
    front_path: params.frontPath,
    back_path: params.backPath,
    user_id: params.userId,
    category: params.category,
  }),
})
```

`fetch(...)` is the browser's built-in HTTP client. `method: 'POST'` is the verb meaning "here is data, act on it and create a result." `headers` carry metadata: `'Content-Type': 'application/json'` announces the body is JSON, and `...(await authHeaders())` spreads in the `Authorization` header carrying the login token (Phase 6). `body: JSON.stringify({...})` converts our JavaScript object into JSON text for transmission. Note we only send the two *filenames* and a category — the backend will fetch the actual images itself.

Finally, the screens that *read* data (dashboard, history) do so with role-based scoping, so an officer only ever sees their own scans:

frontend/src/hooks/useScans.ts

```
let query = supabase
  .from('scans')
  .select('id, created_at, storage_path, extracted, violations, status, user_id, category')
  .order('created_at', { ascending: false })

if (!isAdmin) {
  query = query.eq('user_id', user.id)
}
```

This builds a database query directly from the browser (using the *limited public* key). `.select(...)` names the columns to fetch, `.order('created_at', { ascending: false })` returns newest first, and the key line `if (!isAdmin) query = query.eq('user_id', user.id)` adds a filter: non-admins only get rows where the

owner column equals their own id. Admins skip that filter and see everything. (This is a convenience filter; the database *also* enforces the same scoping itself via RLS in Phase 6, so it can't be bypassed by tampering with the frontend.)

3. Why we did it this way

Uploading photos straight to storage (not through our backend) keeps the backend fast and cheap — it never handles megabytes of image, only tiny filenames. **Role enforcement lives in the router and the queries, not just in hidden buttons**, because a determined user can type a URL by hand; real access control must be in code (and, ultimately, in the database). **React + TypeScript + Vite** is the mainstream, well-documented stack, and TypeScript's type-checking catches shape-mismatch bugs before they ship. **A shared auth context** means every screen agrees on one source of truth about who's logged in, instead of each screen re-deriving it and risking inconsistency.

Phase 3 — The backend API: endpoints and the request lifecycle

1. The concept

A kitchen doesn't cook "anything" — it has a **menu of dishes it knows how to make**. Each menu item is an **endpoint**: a specific address plus a specific action the server performs when that address is called with a specific **HTTP method** (the verb). The two verbs we use:

- **POST** — "here's data; do something and create/return a result." Scanning is a POST.
- **GET** — "just give me something; change nothing." Downloading a notice is a GET.

When a request arrives, it flows through the endpoint's code **in steps**. If a step can't proceed, the server replies with a numbered **HTTP status code** — a standard signal every client understands: 200 = success; 401 = "you're not authenticated"; 403 = "authenticated, but not allowed"; 404 = "not found"; 422 = "your input was malformed"; 429 = "too many requests"; 500/502 = "the server or an upstream service failed." Our backend is built with **FastAPI**, a Python framework whose whole job is to declare these endpoints, validate inputs, run the steps, and format responses.

A FastAPI idea you'll see everywhere: **dependency injection**. Instead of an endpoint fetching its own tools, it *declares what it needs* as parameters, and FastAPI builds and hands them in before running the endpoint. It's like a chef saying "I need a prepped station and a verified customer ticket" and the kitchen guaranteeing those exist before cooking starts. This is how we bolt authentication onto every protected endpoint uniformly.

2. What we actually built

The scan endpoint is declared with a decorator and declares its dependencies as parameters:

backend/app/api/scans.py

```
@router.post("", response_model=ScanResponse, status_code=status.HTTP_200_OK, ...)
def create_scan(
    payload: ScanRequest,
    current_user: Dict[str, Optional[str]] = Depends(get_current_user),
    supabase_service: SupabaseService = Depends(get_supabase_service),
    gemini_service: GeminiService = Depends(get_gemini_service),
) -> ScanResponse:
```

`@router.post("")` registers this function to answer a POST at `/api/scans` (the `/api/scans` prefix is set once when the router is created). `response_model=ScanResponse` tells FastAPI the exact shape it will return, which it validates and documents automatically. The parameters are the **injected dependencies**: `payload: ScanRequest` is the incoming JSON body, automatically parsed and validated against the `ScanRequest` schema — if a caller omits `front_path`, FastAPI rejects the request with 422 before our code even runs. The three `Depends(...)` parameters ask FastAPI to construct, in order, the authenticated user, a database/storage connection, and an AI connection. `get_current_user` is the security guard (Phase 6); because it's a dependency, *this endpoint cannot run for an unauthenticated caller* — the guard raises 401 first.

Inside, the request lifecycle reads like a five-step recipe straight down the function:

`backend/app/api/scans.py`

```
# Owner is derived from the authenticated token, NOT the request body.
user_id = current_user["id"]
_enforce_scan_rate_limit(user_id)

front_path = payload.front_path.strip()
back_path = payload.back_path.strip()
category = payload.category.strip() if payload.category else None

if not front_path or not back_path:
    raise HTTPException(status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
                        detail="Both 'front_path' and 'back_path' must be provided.")

# 1. Fetch BOTH images from Supabase Storage
front_bytes = supabase_service.fetch_image(front_path)
back_bytes = supabase_service.fetch_image(back_path)

# 2. Extract declarations via Gemini in a SINGLE call
front_mime = supabase_service.get_mime_type(front_path)
back_mime = supabase_service.get_mime_type(back_path)
extracted = gemini_service.extract_label_data(
    front_image_bytes=front_bytes, back_image_bytes=back_bytes,
    front_mime_type=front_mime, back_mime_type=back_mime)

# 3. Run the deterministic Legal Metrology checks
violations, compliance_status = check_compliance_rules(extracted, category=category)

# 4. Save record to Supabase 'scans' table
supabase_service.save_scan_record(front_path=front_path, back_path=back_path,
    extracted=extracted.model_dump(), violations=[v.model_dump() for v in violations],
    status=compliance_status, user_id=user_id, category=category)

# 5. Return extracted data and violations
return ScanResponse(extracted=extracted, violations=violations, status=compliance_status)
```

(The real file wraps each external call in `try/except` with specific error codes — shown next — but this is the spine.) The very first lines do two security things *before any work*: `user_id = current_user["id"]` takes the owner from the verified identity (never from the request body — anti-spoofing, Phase 6), and `_enforce_scan_rate_limit` caps how often a user can trigger this expensive path. Then the five steps: **(1)** download both images from storage by filename; **(2)** detect their image types and hand both to the AI in one call, receiving a validated `extracted` object (Phase 4); **(3)** run that through the rulebook to get `violations` and a `compliance_status` (Phase 5); **(4)** persist a permanent record, converting the Pydantic objects to plain dictionaries with `.model_dump()` for storage; **(5)** return the `ScanResponse` to the phone.

The real error handling turns any failure into a precise status code. For example, the image fetch:

`backend/app/api/scans.py`

```
try:
    front_bytes = supabase_service.fetch_image(front_path)
except FileNotFoundError as fnf_err:
    logger.warning(f"Front image not found: {fnf_err}")
    raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                        detail="Front image not found in storage.")
except Exception as exc:
    logger.error(f"Failed to fetch front image from Supabase storage: {exc}")
    raise HTTPException(status_code=status.HTTP_502_BAD_GATEWAY,
```

```
detail="Failed to retrieve the front image from storage.")
```

Two things to notice for the presentation. First, distinct failures map to distinct codes: a *missing* file becomes `404`, a *storage system* failure becomes `502` ("bad gateway" — an upstream service we depend on failed). Second — a deliberate security choice — the message returned to the client is generic ("Failed to retrieve the front image"), while the *detailed* internal error is written only to the server log with `logger.error(...)`. We never leak internal error text (which could reveal system details) to the outside world.

The second endpoint, the notice, is a **GET** that reads an existing record and returns a file:

backend/app/api/scans.py

```
@router.get("/{scan_id}/notice", ...)
def download_improvement_notice(
    scan_id: str,
    current_user: Dict[str, Optional[str]] = Depends(get_current_user),
    supabase_service: SupabaseService = Depends(get_supabase_service),
) -> Response:
    scan = supabase_service.fetch_scan(scan_id)
    if not scan:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Scan not found.")

    # Authorisation: only the scan's owner or an admin may download it.
    if scan.get("user_id") != current_user["id"] and current_user.get("role") != "admin":
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                            detail="You are not authorised to access this record.")
    ...
    return Response(content=pdf_bytes, media_type="application/pdf",
                    headers={"Content-Disposition": f'attachment; filename="{filename}"'})
```

`{scan_id}` in the path is a **path parameter** — the id is part of the URL (`/api/scans/abc123/notice`). After loading the record, it enforces authorisation explicitly: unless you're the owner *or* an admin, you get `403`. Finally it returns a raw `Response` whose `media_type="application/pdf"` and `Content-Disposition: attachment; filename=...` header together tell the browser "this is a PDF file, download it with this name" (Phase 7 builds the bytes).

3. Why we did it this way

A **readable five-step pipeline** (fetch -> extract -> judge -> save -> return) means any teammate — or the legal team — can follow exactly what happens to a scan, in order. **Specialist service objects** (`supabase_service`, `gemini_service`) keep the messy details of talking to storage or the AI out of the endpoint, so the endpoint stays a short story and the specialists can be hardened or swapped independently. **Precise status codes plus generic client messages** give clients enough to react correctly (retry vs re-login vs fix input) without ever exposing internal details to a potential attacker. **Dependency injection for auth** means security is declared once per endpoint and can't be forgotten.

Phase 4 — AI extraction with Gemini (and what JSON / structured output really is)

1. The concept

Step 2 of the recipe is the *only* place we use artificial intelligence, and it does exactly one narrow job: **read the text off a photo of a label**. It's a very good pair of eyes — nothing more. It does not decide whether the product is legal; that's Phase 5's job, on purpose.

The model is Google's **Gemini**, a **multimodal** model — "multimodal" means it accepts more than one kind of input, here images *and* text instructions, and produces text. But free-form text is unusable for a program. If we asked "what's on this label?" it might reply in a friendly paragraph, and reliably pulling specific facts out of prose is brittle and error-prone.

So we require **structured output** in **JSON**. **JSON** (JavaScript Object Notation) is a strict, universal, text-based way to write labeled data that every programming language can read. Think of it as a **fill-in-the-blank form** instead of an essay. Rather than "The MRP looks like about 5 rupees, tax included," JSON is exact:

```
{ "mrp": { "value": "Rs 5.00", "inclusive_of_taxes_stated": true } }
```

Same facts, but every blank has a fixed name and place, and the data types are unambiguous (`true` is a real boolean, not the word "true" in a sentence). We hand Gemini the *shape of the form in advance* — a **schema** — and force it to fill in exactly that form. This is the single most important reliability decision in the AI layer.

2. What we actually built

First we define the form precisely. This uses **Pydantic**, a Python library where you declare the shape of data as a class and it validates real data against it:

backend/app/schemas/scan.py

```
class NetQuantity(BaseModel):
    value: str = Field(description="Net quantity magnitude, e.g., '100', '1.5'")
    unit: str = Field(description="Unit of measurement, e.g., 'g', 'ml', 'kg', 'N'")

class MRP(BaseModel):
    value: str = Field(description="Maximum Retail Price amount, e.g., '45.00', 'Rs 120'")
    inclusive_of_taxes_stated: bool = Field(default=False,
        description="True if 'inclusive of all taxes' or similar phrasing is explicitly stated")

class ExtractedData(BaseModel):
    product_name: Optional[str] = Field(default=None, ...)
    manufacturer_packer_importer: Optional[str] = Field(default=None, ...)
    net_quantity: Optional[NetQuantity] = Field(default=None, ...)
    mrp: Optional[MRP] = Field(default=None, ...)
    mfg_or_pack_date: Optional[str] = Field(default=None, ...)
    consumer_care: Optional[str] = Field(default=None, ...)
    declarations_present: List[str] = Field(default_factory=list, ...)
```

Each attribute is a blank on the form, with its type and a human description. `Optional[str]` with `default=None` means "text, or absent if not on the label." `net_quantity` and `mrp` are *nested* sub-forms — `NetQuantity` insists on both a `value` and a `unit`; `MRP` carries a `value` and a **boolean** `inclusive_of_taxes_stated`. Those `description` texts aren't just comments — they're sent to the model as part of the schema, so the AI knows what each blank means. This one class does triple duty: it's the instruction to the AI, the automatic validator of the AI's reply, and the type the rest of the backend works with.

The AI call sends both images in one request and switches on JSON mode:

backend/app/services/gemini_service.py

```
front_part = types.Part.from_bytes(data=front_image_bytes, mime_type=front_mime_type)
back_part = types.Part.from_bytes(data=back_image_bytes, mime_type=back_mime_type)

config = types.GenerateContentConfig(
    response_mime_type="application/json",
    response_schema=ExtractedData,
    system_instruction=SYSTEM_PROMPT,
)

contents = [
    "Image 1 (Front of product package):", front_part,
    "Image 2 (Back of product package):", back_part,
    "Extract all Legal Metrology declarations across both images of this product according to the schema.",
]
```

`types.Part.from_bytes(...)` wraps each image's raw bytes together with its **MIME type** (the standard label for a file's format, e.g. `image/jpeg`) so the model knows how to decode it. The two settings that enforce structured output: `response_mime_type="application/json"` ("answer only in JSON, no prose") and `response_schema=ExtractedData` ("and it must match this exact form").

`system_instruction=SYSTEM_PROMPT` is the standing brief. The `contents` list literally interleaves captions and image parts so a *single* call sees both sides at once and can combine declarations that are split across them.

The `SYSTEM_PROMPT` encodes domain rules that keep the extraction honest — for example:

backend/app/services/gemini_service.py

```
SYSTEM_PROMPT = """You are an expert Legal Metrology compliance auditor.
...
Extraction Guidelines:
1. `product_name`: The common or generic name of the commodity – its category descriptor, NOT the brand name (e.g. 'Potato Chips', 'Namkeen', 'Toothpaste', 'Biscuits'). Return null if not present.
...
4. `mrp`: ... `inclusive_of_taxes_stated` must be true only if words like 'inclusive of all taxes', 'incl. of all taxes', or 'all taxes included' are explicitly written on the package, false otherwise.
...
"""
```

These guidelines matter legally: guideline 1 stops the model from reporting the *brand* ("Yellow Diamond") as the product name when the law wants the *generic* name ("Namkeen"); guideline 4 stops it from *assuming* prices include tax — it may only set that flag if the exact words appear. The AI reads the label faithfully; it does not editorialize.

Finally, we validate the AI's reply back into our form:

backend/app/services/gemini_service.py

```
raw_text = response.text
if not raw_text:
    logger.warning("Gemini returned empty response text")
    return ExtractedData()

extracted = ExtractedData.model_validate_json(raw_text)
return extracted
```

`model_validate_json(raw_text)` parses the returned JSON *and checks it against the schema* — if Gemini ever produced a malformed or mistyped reply, this raises rather than letting corrupt data flow onward. If the model returns nothing, we hand back an empty `ExtractedData()` — which, importantly, the rule engine will correctly flag as non-compliant (all mandatory fields missing), so an AI failure degrades to "flagged," never to a false "compliant."

3. Why we did it this way

Forcing JSON/structured output is what makes the downstream rulebook possible — a deterministic checklist can only run on clean, predictably-shaped data. **One call with both images** lets the model merge declarations spread across front and back (MRP on front, manufacturer on back) and costs one AI charge instead of two. **The AI only reads, never judges** because AI answers can be subtly wrong or vary between runs, and a legal verdict must be exact, repeatable, and explainable — that's deterministic code's job. **Validating the reply with the same schema** turns the AI from an unpredictable black box into a component with a guaranteed output contract; anything off-contract is caught immediately. **Choosing the `flash-lite` model** (set in config; Phase 8) gives us higher request limits and lower latency for this read-only task, at the cost of raw power we don't need for OCR-style extraction.

Phase 5 — The deterministic rule engine (the 8 real rules, in full)

1. The concept

Now we *judge*. **Deterministic** means "the same input always produces exactly the same output, with no randomness or guessing" — the opposite of an AI, and precisely what the law requires: two officers scanning the same label must get the identical verdict, and we must be able to point at *which rule* was broken and *why*.

Picture a human inspector with a **clipboard checklist**: "Maker's name printed? [yes]. Net weight in proper units like grams — not 'a dozen'? [yes]. Price marked as inclusive of tax? [no] — write it up." Our rule engine is that clipboard turned into plain, boring, predictable code. It reads the form the AI filled in (Phase 4) and, for each rule, asks one yes/no question. Every "no" becomes a **violation** carrying the exact legal reference.

The rules come from India's **Legal Metrology (Packaged Commodities) Rules, 2011** — the law governing what must be printed on any pre-packaged good offered for sale.

2. What we actually built

The whole engine is one file, `backend/app/rules/engine.py`, and it opens with a note that this is deterministic Python with no AI, plus instructions for non-programmers on how to edit it. The rules themselves are a plain **list of dictionaries** — deliberately data, not buried logic:

`backend/app/rules/engine.py`

```
RULES: List[dict] = [
    {"field": "manufacturer_packer_importer", "rule_ref": "Rule 6(1)(a)",
     "issue": "Name and address of manufacturer/packer/importer missing", "check": _has_manufacturer},
    {"field": "product_name", "rule_ref": "Rule 6(1)(b)",
     "issue": "Common or generic name of the commodity not declared", "check": _has_product_name},
    {"field": "net_quantity", "rule_ref": "Rule 6(1)(c)",
     "issue": "Net quantity missing or not in standard metric units", "check": _net_quantity_standard},
    {"field": "mfg_or_pack_date", "rule_ref": "Rule 6(1)(d)",
     "issue": "Month and year of manufacture/packing/import not declared", "check": _has_mfg_date},
    {"field": "mrp", "rule_ref": "Rule 6(1)(e)",
     "issue": "Retail sale price (MRP) not declared", "check": _has_mrp},
    {"field": "mrp", "rule_ref": "Rule 2(m)",
     "issue": "MRP not declared as inclusive of all taxes", "check": _mrp_tax_inclusive},
    {"field": "consumer_care", "rule_ref": "Rule 6(2)",
     "issue": "Consumer care details (name, address, phone, email) missing", "check":
     _has_consumer_care},
    {"field": "net_quantity", "rule_ref": "Rule 13(4)",
     "issue": "Net quantity uses a non-standard unit (e.g. dozen/score/gross)", "check":
     _net_quantity_not_prohibited_unit},
]
```

Each entry has: `field` (which extracted field it concerns), `rule_ref` (the exact law, e.g. Rule 6(1)(a)), `issue` (the plain-English sentence that goes on the notice), and `check` (a tiny function returning `True` when the label passes). The eight rules, in order: **6(1)(a)** maker's name & address, **6(1)(b)** the product's generic name, **6(1)(c)** net quantity in a standard unit, **6(1)(d)** manufacture/pack date, **6(1)(e)** the MRP, **2(m)** MRP declared inclusive of all taxes, **6(2)** consumer-care contact details, and **13(4)** the quantity unit isn't a

forbidden one.

The check functions are deliberately tiny and legible. A simple presence check:

backend/app/rules/engine.py

```
def _has_manufacturer(e: ExtractedData) -> bool:
    # Rule 6(1)(a)
    return _text(e.manufacturer_packer_importer)
```

`_text(...)` is a helper meaning "is this a real, non-empty, non-whitespace string?" So this rule passes only if a manufacturer string is actually present. The unit rule is a little richer:

backend/app/rules/engine.py

```
def _net_quantity_standard(e: ExtractedData) -> bool:
    # Rule 6(1)(c) – present, has a value, and unit is a standard metric unit.
    nq = e.net_quantity
    if nq is None or not _text(nq.value):
        return False
    return _canonical_unit(nq.unit) in STANDARD_UNITS
```

"No quantity, or no numeric value -> fail. Otherwise, is the (tidied) unit a real metric unit?" Real labels spell units inconsistently (g, GM, gms, grams), so `_canonical_unit` normalizes spelling before checking, using three editable vocabularies:

backend/app/rules/engine.py

```
STANDARD_UNITS = {"g", "kg", "mg", "ml", "l", "cm", "m", "n", "u"}

UNIT_ALIASES = {
    "gm": "g", "gms": "g", "gram": "g", "grams": "g",
    "kgs": "kg", "kilogram": "kg", "kilograms": "kg",
    "mgs": "mg", "mls": "ml", "milliliter": "ml", "millilitre": "ml",
    "ltr": "l", "ltrs": "l", "litre": "l", "liter": "l", "litres": "l", "liters": "l",
    "nos": "n", "no": "n", "unit": "u", "units": "u", "count": "u",
}

NON_STANDARD_UNITS = {
    "dozen", "dozens", "doz", "score", "scores",
    "gross", "grosses", "piece", "pieces", "pcs", "pc",
}
```

`STANDARD_UNITS` is the allow-list of legal metric units (plus n/u for a plain count). `UNIT_ALIASES` folds common spellings onto the canonical form, so `GM` is accepted as `g`. `NON_STANDARD_UNITS` is the explicit *forbidden* list that Rule 13(4) rejects. {...} here is a Python **set** — an unordered collection with instant membership tests (`unit in STANDARD_UNITS`). Crucially, a non-programmer on the legal team can add a spelling or a forbidden term here without touching any logic.

Now a subtle correctness detail — **not double-firing**. If a label has *no MRP at all*, we should raise exactly one violation ("MRP not declared"), not also a nonsensical second one ("MRP not marked tax-inclusive"). So the tax rule *excuses itself* when there's no MRP:

backend/app/rules/engine.py

```
def _mrp_tax_inclusive(e: ExtractedData) -> bool:
    # Rule 2(m) – MRP must be declared inclusive of all taxes.
    # Only meaningful when an MRP exists; if MRP is missing, Rule 6(1)(e) already
    # covers it, so skip (pass) here to avoid double-firing.
    if not _has_mrp(e):
        return True
    return bool(e.mrp.inclusive_of_taxes_stated)
```

`if not _has_mrp(e): return True` means "no MRP? not my department — pass." The forbidden-unit rule (`_net_quantity_not_prohibited_unit`) uses the same guard: it passes when there's no quantity at all, because the missing-quantity rule already covers that case. This is why an *empty* extraction produces exactly six violations (the six presence rules), not eight — the two conditional rules correctly stand down.

The engine driver runs the checklist and computes the verdict:

backend/app/rules/engine.py

```
def check_compliance_rules(extracted, category=None):
    violations: List[Violation] = []
    applicable_rules = RULES + CATEGORY_RULES.get(category or "", [])

    for rule in applicable_rules:
        passed = rule["check"](extracted)
        if not passed:
            violations.append(Violation(field=rule["field"], issue=rule["issue"],
            rule_ref=rule["rule_ref"]))

    status = "flagged" if violations else "compliant"
    return violations, status
```

It walks every applicable rule; each `False` becomes a `Violation` recording the field, the sentence, and the law. The verdict is then trivial and unarguable: **any violations -> flagged; none -> compliant**.

`applicable_rules = RULES + CATEGORY_RULES.get(category or "", [])` shows a deliberate design seam — a hook for per-category rules — but `CATEGORY_RULES` is an empty dictionary today:

backend/app/rules/engine.py

```
# Every product category currently runs the EXACT SAME 8 Legal Metrology rules
# above. This mapping is intentionally EMPTY today ... Do NOT add FSSAI or other
# non-Legal-Metrology rules here.
CATEGORY_RULES: Dict[str, List[dict]] = {}
```

So every category (Food & Beverages, Textiles, etc.) runs the identical eight checks — the category is recorded on the scan for classification, but it does not change the verdict.

3. Why we did it this way

The AI extracts but this Python engine judges because a legal verdict must be deterministic, explainable, and auditable — if a manufacturer disputes a violation we can point at Rule 6(1)(c) and the exact code that raised it, which is impossible with an AI opinion. **Rules stored as a plain list with plain-English sentences** let the legal team read, correct, and extend the rulebook without being programmers. **The double-fire guards** ensure a fair notice lists each real problem once, not redundant complaints. **FSSAI is explicitly excluded** because FSSAI is India's *food-safety* regulator — a different body from Legal Metrology, which

governs *packaging declarations*; mixing them would produce legally wrong notices, so the code's own comment forbids it. **The empty CATEGORY_RULES seam** documents where future category-specific Legal Metrology rules would slot in, without adding any today.

Phase 6 — Data, authentication & security

1. The concept

Three big ideas live here.

A **database** is an organized, permanent filing cabinet. Instead of paper it holds **tables** (like spreadsheets); each **row** is one record and each **column** a field. We use **Supabase**, a platform that bundles a **PostgreSQL** database (a powerful, decades-proven database engine), file storage, and a login system. We keep two tables: `scans` (one row per inspection) and `profiles` (one row per person, holding their role and status). A special column is often the **primary key** — a value that uniquely identifies a row (here, a UUID).

Authentication ("authn") is *proving who you are* — showing ID at a door. **Authorization** ("authz") is the next question: *given who you are, what may you do?* When you sign in with Google, Supabase issues your browser a **token**, specifically a **JWT** (JSON Web Token). A JWT is a tamper-proof digital wristband: it's a small piece of JSON (your user id, email, an expiry time) that is **cryptographically signed** by Supabase. Anyone can *read* it, but nobody can *forge or alter* it without Supabase's secret signing key — like a hologram on a passport. Your browser presents this wristband on every backend request; the backend verifies the signature is genuine and unexpired.

Row Level Security (RLS) is a rule enforced by the database *itself* about which rows each user may see or change — imagine a filing cabinet whose lock inspects your ID before letting you open a specific drawer. This is what makes our inspection records **immutable** (unchangeable): the database is configured so that *no logged-in user* can update or delete a scan — not even an admin. A final key concept: the difference between the **anon key** (a limited public key the frontend uses, always subject to RLS) and the **service-role key** (a master key the backend uses that *bypasses* RLS). Because the master key bypasses the database's locks, the backend must do its *own* authorization checks — which is exactly why Phase 3's endpoints re-verify ownership.

2. What we actually built

The tables and their locks are defined in one SQL file. The `profiles` table:

`supabase/schema.sql`

```
create table if not exists public.profiles (  
  id uuid primary key references auth.users (id) on delete cascade,  
  email text,  
  full_name text,  
  role text not null default 'none' check (role in ('admin', 'officer', 'none')),  
  status text not null default 'active' check (status in ('active', 'inactive')),  
  created_at timestampz not null default now()  
);
```

`id uuid primary key references auth.users (id)` means each profile's id is the Supabase login id it belongs to (a **foreign key** linking the two), and `on delete cascade` means deleting the login automatically deletes the profile. The `check (role in ('admin', 'officer', 'none'))` constraint makes it *impossible* to store an invalid role — the database rejects anything else. `status` similarly can only be `active` or `inactive`.

The immutability rule for scans — the security heart of the project:

supabase/schema.sql

```
-- scans: officers read only their own; admins read all.
-- Deliberately NO insert/update/delete policies for clients: inspection
-- records can only be written by the backend (service-role key bypasses RLS)
-- and can never be altered or removed by any user.
alter table public.scans enable row level security;

create policy scans_select_own_or_admin on public.scans
  for select using (auth.uid() = user_id or public.is_admin());
```

`enable row level security` turns the drawer-locks *on* for the scans table. Then we create exactly one **policy**, and only for `select` (reading): `using (auth.uid() = user_id or public.is_admin())` means a logged-in user may read a scan row *only if* their own id (`auth.uid()`) matches that row's `user_id`, **or** they're an admin. There is deliberately **no** policy for `insert`, `update`, or `delete` — with RLS on and no policy granting them, those actions are denied to every client. The only entity that can write scans is our backend, whose master key bypasses RLS. That combination — "read-only, scoped, for clients; writes only via the trusted backend" — is precisely what "immutable inspection record" means, enforced by the database itself rather than by our hoping the app behaves.

"Am I an admin?" is answered by a helper function, written carefully to avoid a classic trap:

supabase/schema.sql

```
create or replace function public.is_admin()
returns boolean language sql stable security definer set search_path = public
as $$
  select exists (
    select 1 from public.profiles
    where id = auth.uid() and role = 'admin' and status = 'active'
  );
$$;
```

This returns true if the current user has an active admin profile. The important keyword is `security definer`: it runs with the *function author's* permissions, which lets it read profiles **without** itself triggering the profiles RLS policies — otherwise those policies would call `is_admin()` which would re-check profiles which would call `is_admin()` ... an infinite loop. `security definer` breaks that recursion cleanly. (`set search_path = public` is a standard safety hardening for such functions.)

Roles are assigned automatically the first time someone signs in, by a database **trigger** — code the database runs by itself when an event happens (here, a new user being created):

supabase/schema.sql

```
if user_email = any (admin_emails) then
  derived_role := 'admin';
elsif user_email like '%@ves.ac.in' then
  derived_role := 'officer';
else
  derived_role := 'none'; -- signed in, but no access until an admin grants a role
end if;

insert into public.profiles (id, email, full_name, role, status)
values (new.id, new.email, ..., derived_role, 'active')
```

```
on conflict (id) do nothing;
```

In plain English: if the new user's email is on the admin allow-list -> admin; if it ends in @ves.ac.in -> officer; otherwise -> none (signed in but blocked until promoted). `on conflict (id) do nothing` makes it safe to re-run — it won't overwrite an existing profile (so an admin's later promotion of someone isn't undone on their next login).

Storage has its own lock, allowing only logged-in users to upload evidence:

supabase/schema.sql

```
create policy evidence_upload_authenticated on storage.objects
  for insert to authenticated
  with check (bucket_id = 'evidence-photos');
```

`for insert to authenticated` permits uploads only by signed-in users, and `with check (bucket_id = 'evidence-photos')` restricts them to our one bucket.

On the **frontend**, the shared auth context (from Phase 2) tracks login state and derives the two facts the whole app keys off:

frontend/src/auth/AuthContext.tsx

```
const { data: sub } = supabase.auth.onAuthStateChange((_event, newSession) => {
  setSession(newSession)
  setLoadingSession(false)
  if (newSession?.user) {
    void loadProfile(newSession.user.id)
  } else {
    setProfile(null)
  }
})
```

`onAuthStateChange(...)` is Supabase notifying us whenever login state changes (sign-in, sign-out, token refresh). We store the `session` (which contains the JWT and auto-refreshes) and load the matching profile (which contains role/status). From these, the context computes `accessDenied` (logged in but not an active officer/admin) and `isAdmin` — the exact flags Phase 2's routes and nav consulted. `loadProfile` even retries a few times, because right after a brand-new sign-up the trigger's freshly-created profile row can take a moment to be readable.

On the **backend**, the security guard that protects every endpoint — this is the fix that made the kitchen safe once it went public:

backend/app/api/scans.py

```
def get_current_user(authorization: Optional[str] = Header(default=None),
                     supabase_service: SupabaseService = Depends(get_supabase_service)):
    if not authorization or not authorization.lower().startswith("bearer "):
        raise HTTPException(status_code=401, detail="Authentication required.")
    token = authorization[7:].strip()
    user = supabase_service.get_user_from_token(token)
    if not user:
        raise HTTPException(status_code=401, detail="Invalid or expired session. Please sign in again.")
```

```

profile = supabase_service.fetch_profile(user["id"])
if (not profile or profile.get("status") != "active"
    or profile.get("role") not in ("officer", "admin")):
    raise HTTPException(status_code=403, detail="Your account is not authorised.")
return {"id": user["id"], "email": user.get("email"), "role": profile.get("role")}

```

Step by step: the login wristband arrives in the Authorization header as Bearer <token>. No header, or one not starting with bearer -> **401**. We strip off "Bearer " (7 characters) and validate the token; if Supabase says it's fake or expired -> **401**. Then we load the user's profile; if they aren't an *active* officer/admin -> **403**. Only a genuine, authorised user passes, and we return their id, email, and role for the endpoint to use.

The actual token validation lives in the storage/database service and asks Supabase to verify the signature:

backend/app/services/supabase_service.py

```

def get_user_from_token(self, token: str) -> Optional[Dict[str, Any]]:
    if not token:
        return None
    try:
        resp = self.client.auth.get_user(token)
    except Exception as exc:
        logger.info(f"Access token validation failed: {exc}")
        return None
    user = getattr(resp, "user", None)
    if not user or not getattr(user, "id", None):
        return None
    return {"id": user.id, "email": getattr(user, "email", None)}

```

`self.client.auth.get_user(token)` hands the wristband to Supabase, which checks its signature and expiry and returns the user it belongs to (or errors if it's invalid). We wrap it in `try/except` so any bad token cleanly becomes `None` (-> a **401** upstream) rather than crashing.

Two more hardening touches you can point to in the demo. **Anti-spoofing** — the owner is taken from the verified identity, never from what the client claims:

backend/app/api/scans.py

```

# Owner is derived from the authenticated token, NOT the request body – a
# client cannot attribute a scan to another user.
user_id = current_user["id"]

```

And a **rate limit** on the AI-billed endpoint, so no single user can hammer it:

backend/app/api/scans.py

```

def _enforce_scan_rate_limit(user_id: str) -> None:
    now = time.time()
    cutoff = now - _RATE_LIMIT_WINDOW
    calls = _SCAN_CALLS[user_id]
    calls[:] = [t for t in calls if t > cutoff]
    if len(calls) >= _RATE_LIMIT_MAX:
        raise HTTPException(status_code=429,
            detail="Too many scans in a short time. Please wait a moment and try again.")
    calls.append(now)

```

It keeps each user's recent scan timestamps in memory, discards ones older than the window (`_RATE_LIMIT_WINDOW`, 60 seconds), and if there are already too many (`_RATE_LIMIT_MAX`, 20) returns 429 "slow down." Otherwise it records this call and proceeds.

One last real-world detail worth knowing: the record-saving function tries a full write first and *falls back* if the database schema is older than expected:

`backend/app/services/supabase_service.py`

```
try:
    response = self.client.table("scans").insert(full_payload).execute()
    ...
except Exception as exc:
    err_msg = str(exc)
    if "PGRST204" in err_msg or "column" in err_msg.lower():
        # Fallback if front_path / back_path columns are not yet added
        fallback_payload = {"storage_path": combined_storage_path,
                           "extracted": extracted, "violations": violations, "status": status}
        if user_id: fallback_payload["user_id"] = user_id
        if category: fallback_payload["category"] = category
        response = self.client.table("scans").insert(fallback_payload).execute()
```

It first tries to store separate `front_path/back_path` columns; if the live table doesn't have them (a "column not found" error), it retries writing just the combined `storage_path` (formatted as `"front.jpg | back.jpg"`). That resilience is why the app kept working across database revisions — and it's why other code (like the PDF's `_split_storage_path`) knows how to split that combined string back into two filenames.

3. Why we did it this way

Immutability is enforced at the database, not just by hiding buttons, because inspection evidence must be trustworthy in a dispute — if anyone (or a bug in our own code) could quietly edit a past verdict, the system is worthless. RLS with "read-only, scoped, no write policy for clients" guarantees that at the lowest level. **The backend re-checks authorization** (the guard, anti-spoofing, owner-or-admin) because its master key bypasses RLS — without those checks it would be an open door. **Roles are derived from email automatically** so `@ves.ac.in` officers get in on first login with zero setup while everyone else is safely locked out until promoted. **security_definer_on_is_admin()** is a specific, necessary technique to avoid infinite RLS recursion. **The rate limit and generic error messages** are defense-in-depth: limiting abuse of a paid endpoint, and never leaking internal details to a potential attacker.

Phase 7 — The PDF Improvement Notice

1. The concept

An inspection isn't much use if the officer can't hand over an official piece of paper. So any saved scan can be turned into a formal, printable **PDF** — a "Legal Metrology Improvement Notice."

The clean way to produce a good-looking document is a **template**: a pre-designed page with blanks, like an official letterhead where the date, recipient, and body are left empty to be filled per case. We design the notice once as a web page (**HTML** for structure, **CSS** for styling) full of blanks, then for each scan we fill the blanks with that scan's real data and convert the finished page into a PDF. The filling-in is done by **Jinja2** (a templating engine that substitutes values into `{{ placeholders }}`), and the HTML->PDF conversion by **xhtml2pdf** (a pure-Python library). One more concept: to embed the photos without linking to external files, we encode each image as a **data URI** — the image bytes written directly into the HTML as text (base64), so the document is fully self-contained.

2. What we actually built

The whole notice lives in `backend/app/services/report_service.py`. The template is an HTML string; its styling is defined once at the top:

backend/app/services/report_service.py

```
@page { size: a4 portrait; margin: 1.4cm 1.5cm; }
body { font-family: Helvetica, Arial, sans-serif; color: #1a1c1e; font-size: 10pt; }
.bar { background-color: #002045; color: #ffffff; padding: 12pt 14pt; }
...
.verdict.compliant { background-color: #166534; }
.verdict.flagged { background-color: #991b1b; }
...
.vtable th { background-color: #002045; color: #ffffff; ... }
.avoid-break { -pdf-keep-with-next: true; page-break-inside: avoid; }
```

`@page { size: a4 portrait; margin: ... }` sets a real printable A4 page. The colour classes encode the institutional look: a navy header bar (`.bar, #002045`), and a verdict banner that is green

(`.verdict.compliant, #166534`) or red (`.verdict.flagged, #991b1b`) depending on the outcome.

`.avoid-break` uses `page-break-inside: avoid` to keep the evidence section and its photos together on one page rather than splitting across a page boundary — a real layout fix we added after seeing the photos orphan onto the next page.

The main function gathers the scan's real data into a `context` — the bag of values that fill the blanks:

backend/app/services/report_service.py

```
now = datetime.now(timezone.utc)
context = {
    "notice_ref": str(scan.get("id", "-")),
    "notice_date": now.strftime("%d %b %Y"),
    "inspection_date": _fmt_dt(scan.get("created_at"), with_time=True),
    "officer_name": officer_name or "Unknown officer",
    "category": scan.get("category") or "General",
```

```

    "packer": extracted.get("manufacturer_packer_importer") or "Not declared on package",
    "product_name": extracted.get("product_name"),
    "net_quantity": net_quantity_text,
    ...
}

```

Every value comes straight from the immutable scan record: the scan's `id` becomes the notice's reference number, the extracted manufacturer becomes who the notice is "addressed to," the inspection timestamp is formatted for humans, and so on. Nothing is invented — the notice is a faithful printout of the stored record.

The evidence photos need care, because a tall phone photo would overflow the page. Before embedding, each is downscaled and fitted into a fixed box, and we compute its exact display size:

backend/app/services/report_service.py

```

    im = Image.open(io.BytesIO(raw))
    im = im.convert("RGB")
    im.thumbnail((max_px, max_px)) # in-place, keeps aspect ratio
    buf = io.BytesIO()
    im.save(buf, format="JPEG", quality=82)
    uri = "data:image/jpeg;base64," + base64.b64encode(buf.getvalue()).decode("ascii")

    w_px, h_px = im.size
    aspect = (w_px / h_px) if h_px else 1.0
    w_cm = max_w_cm
    h_cm = w_cm / aspect
    if h_cm > max_h_cm:
        h_cm = max_h_cm
        w_cm = h_cm * aspect
    return uri, round(w_cm, 2), round(h_cm, 2)

```

Using the **Pillow** image library: `Image.open(io.BytesIO(raw))` reads the raw bytes; `.convert("RGB")` normalizes the colour format; `.thumbnail((max_px, max_px))` shrinks it *in place while preserving its proportions* (never stretching); `.save(buf, "JPEG", quality=82)` re-encodes it smaller. Then `base64.b64encode(...)` turns those bytes into the text **data URI** the HTML embeds. The final block is the aspect-ratio maths: start at the max box width, compute the height that keeps the photo's proportions, and if that height overflows the box, pin the height to the max and back-compute a narrower width instead. Returning an explicit width **and** height in centimetres is what guarantees a portrait photo can never blow up the layout — the earlier bug, fixed for good.

Finally, fill the template and convert to PDF bytes:

backend/app/services/report_service.py

```

    html = _template.render(**context)

    buffer = io.BytesIO()
    result = pisa.CreatePDF(src=html, dest=buffer, encoding="utf-8")
    if result.err:
        logger.error("xhtml2pdf reported %s error(s) generating the notice", result.err)
        raise RuntimeError("Failed to generate the improvement notice PDF.")
    return buffer.getvalue()

```

`_template.render(**context)` is Jinja2 pouring our real values into every `{{ blank }}`, producing a finished HTML page. `pisa.CreatePDF(src=html, dest=buffer)` is `xhtml2pdf` converting that page into PDF bytes written into an in-memory buffer. If conversion reports any error we raise (and the endpoint returns 500);

otherwise `buffer.getvalue()` is the finished PDF, which Phase 3's endpoint streams back to the phone as a download.

3. Why we did it this way

A template rather than drawing the PDF by hand because designing a page in HTML/CSS is far easier to build and maintain than positioning every element of a PDF manually — edit the template and every future notice updates. **xhtml2pdf over the more famous WeasyPrint** because WeasyPrint needs heavy system libraries (Cairo/Pango) that are painful to install, especially on Windows where the team develops and on free hosting; xhtml2pdf is pure Python and simply works everywhere. **Pillow pre-sizing with explicit dimensions** because real evidence photos are tall and multi-megabyte, and without fitting them into a fixed box they overflowed the page — pre-sizing keeps the layout clean and the file small. **Generated from the immutable record** so the notice always reflects exactly what was inspected, never an editable re-interpretation.

Phase 8 — Deployment: putting it live on the internet

1. The concept

Deployment means moving the code off a laptop and onto always-on computers on the internet, so anyone with the link can use it. Each of our two programs goes to a specialist **host** (a company that runs your code on their machines): the frontend to **Vercel** (excellent at serving websites) and the backend to **Render** (good at running Python servers). The database/storage/login already live on Supabase.

Two supporting concepts finish the picture. **Environment variables** are settings fed to a program from the outside rather than written into the code — used for secrets (keys) and for values that differ between "my laptop" and "the real internet" (like the backend's public URL). Keeping secrets in environment variables means they never sit in the source code where they could leak. And **CORS** — the browser permission from Phase 1 — must now name the real production frontend address, or the live site can't call the live backend. Getting environment variables and CORS right is most of what "wiring a deployment" means.

2. What we actually built

The backend's deployment recipe is written down so it's reproducible:

`render.yaml`

```
services:
- type: web
  name: parakhmitra-backend
  runtime: python
  rootDir: backend
  plan: free
  buildCommand: pip install -r requirements.txt
  startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
  healthCheckPath: /health
```

This tells Render: run a Python **web service** whose code is in the `backend` folder; **build** it by installing its dependencies (`pip install -r requirements.txt` — `requirements.txt` lists every Python library the backend needs); **start** it with `uvicorn app.main:app ...` (**uvicorn** is the server engine that actually runs our FastAPI app). `--host 0.0.0.0` means "accept connections from anywhere" (necessary in a data center), and `--port $PORT` uses the port number Render supplies via that environment variable. `healthCheckPath: /health` is the address Render pings to confirm the app is alive — the very endpoint we built in Phase 1.

The secrets (Supabase master key, Gemini key) and the allowed frontend address are **not** in the code — they're set as environment variables in Render's dashboard, and the backend reads them at startup:

`backend/app/config.py`

```
def _parse_cors_origins() -> List[str]:
    raw = os.getenv("CORS_ORIGINS", "").strip()
    if raw:
        return [origin.strip() for origin in raw.split(",") if origin.strip()]
    return ["http://localhost:5173", "http://127.0.0.1:5173"]
```

`os.getenv("CORS_ORIGINS", "")` reads the allowed-origins setting from the environment. If it's set (in production, to our Vercel URL), it splits it on commas into a list of allowed dining rooms; if it's unset (on a laptop), it falls back to the local development addresses. Whatever this returns becomes `settings.CORS_ORIGINS`, which is exactly the guest-list handed to the CORS middleware in Phase 1. So this one environment variable is the switch that lets the live website talk to the live backend. The AI model is configured the same way:

backend/app/config.py

```
GEMINI_MODEL: str = Field(
    default_factory=lambda: os.getenv("GEMINI_MODEL", "gemini-3.5-flash-lite")
)
```

The model name is read from the environment, defaulting to `gemini-3.5-flash-lite` — so we can change or roll back the AI model in the dashboard without editing code.

The frontend needs one deployment rule so that refreshing a deep link doesn't break:

frontend/vercel.json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

`"rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]` says "for *any* address, serve the app's single HTML page, and let the frontend's router (Phase 2) decide which screen to show." This is required for a **single-page application**: without it, reloading the browser on `/history` would ask Vercel for a `/history` file that doesn't exist and return a 404.

How the two live URLs wire together (the process captured in `DEPLOY.md`): deploy the *backend first* to get its URL (ours became `https://sih-d9a.onrender.com`); give that URL to the frontend as the `VITE_API_BASE_URL` environment variable (the address constant from Phase 2); deploy the frontend to get *its* URL (`https://parakh-mitra.vercel.app`); then set the backend's `CORS_ORIGINS` to that frontend URL, and add the same URL to Supabase's allowed **redirect URLs** so Google login returns to the live site instead of localhost. At that point the dining room and the kitchen know each other's addresses and permit each other, and the round trip works end-to-end.

3. Why we did it this way

Two specialist hosts because each is best-in-class and free for its job — Vercel for static sites, Render for Python servers — and it keeps the two halves independently deployable. **Environment variables for keys and URLs** so the identical code runs unchanged on a laptop and in production, and so secrets never live in the source. **A checked-in `render.yaml` and `vercel.json`** make the deployment reproducible and self-documenting rather than a pile of manual dashboard clicks nobody remembers.

One real operational quirk to know for the demo: on the **free tier**, **Render puts the backend to sleep after ~15 minutes of no traffic**. The first request after it sleeps has to wake it (~50 seconds), and on a phone that can surface as a one-off "failed to fetch." Hitting `/health` just before presenting — or running a small uptime pinger that calls `/health` every few minutes — keeps it awake. This is a cost trade-off of the free plan, not a

bug; a paid instance never sleeps.

The whole story, in one paragraph

An officer opens the **frontend** (Phase 2) and photographs a package. The two photos go straight to online **storage**; only their filenames are POSTed to the **backend API** (Phase 3). The backend verifies the officer's **login wristband (JWT)** and their role (Phase 6), downloads the photos, and asks **Gemini** to read the label into a strict **JSON form** it validates (Phase 4). A **deterministic rule engine** judges that form against the eight real Legal Metrology rules and returns a verdict with exact legal references (Phase 5). The result is saved as an **immutable record** the database itself won't let anyone edit or delete (Phase 6) and shown back on the phone. Any record can be rendered into an official **PDF notice** built from a Jinja2 template and xhtml2pdf (Phase 7). The whole thing runs live across **Vercel + Render + Supabase**, wired together by environment-variable URLs and CORS permissions (Phase 8) — two programs talking like a dining room and a kitchen (Phase 1).